

Programming 1

Orga

- MidTerm:
 - Dec 16th 2025
 - Finalized by Board of Examination
 - no alternative date
 - No retake on the midterm
 - In presence, pen and paper, no support materials
 - see announcement on ILearn
- Any questions?

Orga

- Recognition requests:
Can be granted in exceptional cases
 - Provide proof that 100% of this courses content have been covered
 - Provide full details e.g. course slides, etc – overviews and abstracts are NOT sufficient
 - Lack of proof causes automatic and final rejection
- Any questions?

Orga: Eval AIN

Information
Technology II (IM-B):

[https://eva.th-deg.de/
evasys/online.php?
typ=html&user_tan=A
66KX](https://eva.th-deg.de/evasys/online.php?typ=html&user_tan=A66KX)



Orga

- We all have a solid understanding of
 - variables, types and collections?
 - Loops
 - conditions
- Any questions so far?

Calendar

6./7.10.	Intro, Orga, Hello, World!	8./9.12.	lectures
13./14.10.	Variables, basics	15./16.12.	mid-term
20.21./10.	Collections, Lists, ...	22./23.12.	lectures
27./28.10.	Conditions and Loops	12./13.1.	lectures
3./4.11.	Functions and Modules	18./19.1.	recap
10./11.11.	Classes and Methods	from 26.1.	exams...
17./18.11.	UI + TK basics		
24./25.11.	Spaceship		
01./02.12.	Invaders and Collisions		

Functions, Modules, I/O operations

Functions

What is it?

Functions

What is it?

- named block of code

```
def greet(name):  
    print(f"Hello, {name}!")  
  
greet("Alice")  
greet("Bob")
```

Functions

What is it?

- named block of code
- **def** means “define a function.”
- **greet** is the function’s name.
- **name** is a **parameter** — a variable the function needs to work.
- Everything indented under **def** runs when you call **greet(...)**

```
def greet(name):  
    print(f"Hello, {name}!")  
  
greet("Alice")  
greet("Bob")
```

```
Hello, Alice!  
Hello, Bob!
```

Functions

Why? Main reasons to use functions:

- Reusability of code
- Organize and reduce the program size
- Functions make a program more readable and understandable
- Makes program management and maintenance easier
- Improves composability, testability, etc..

→ A function is a subprogram that acts on data and may or may not return a value

Example

Without function

```
print("Area of rectangle:")
width = 5
height = 3
area = width * height
print(area)
```

Using function

```
def rectangle_area(width, height):
    return width * height

print("Area of rectangle:", rectangle_area(5, 3))
```

Functions

- The statements indented below the function define the working of the function. This block is also called the body of the function
- The return statements return a result that can be operated on by the user. The user can choose to display it or modify it before displaying it by assigning it to a variable
- The non indented lines below the function are not part of the function. In this case it's the “`print(addition(2,3))`” statement.
- It is simply asking the function to perform the operation on the numbers 2,3 (= call statement)

Arguments vs Parameters

- Parameters
 - Variables that are listed inside the parentheses in a function definition
 - They act as place holders for the values that will be given/passed into the function
 - Exist only inside the function
- Arguments
 - The actual values passed to a function when it is called
 - They get assigned to the corresponding parameters
 - They can exist outside the functions

Arguments vs Parameters

- Key difference :
 - Parameters → what the function expects (placeholders)
 - Arguments → what the function is actually given by the user
- Analogy :
 - Parameters → blank fields in a form (Name : _____)
 - Arguments → filled out fields in a form (Name: Max)

Arguments vs Parameters

- In the example below:
 - Number1 , number2 are parameters
 - 2,3 are the arguments
 - It could've also been set to a default by displaying it as
 - def addition(Number1 = 2, Number2 = 3) :

```
def addition(number1, number2): #def <Function Name> (arguments or parameters):
    result = number1 + number2 #Function body
    return result #return statements return a result
```

```
print(addition(2,3)) # OUTPUT : 5
```


Types of arguments

- Positional Arguments
 - Passed in the order of the parameters
 - Most common and simplest type

```
def greet(name, age):  
    print(f"Hello {name}, you are {age} years old.")
```

```
greet("Alice", 20)  
# "Alice" goes to 'name', 20 goes to 'age'
```

Types of Arguments

- Keyword Arguments
 - Passed by explicitly naming the parameter
 - Order doesn't matter as we are explicitly naming the parameter

```
def greet(name, age):  
    print(f"Hello {name}, you are {age} years old.")
```

```
greet(age=25, name="Bob")  
# Even though order is swapped, works fine  
# "Alice" goes to 'name', 20 goes to 'age'
```

Types of Arguments

- Default Arguments
 - User can assign default values to parameters
 - If no argument is provided, the default is used

```
def greet(name, age=18):  
    print(f"Hello {name}, you are {age} years old.")  
  
greet("Charlie")      # uses default age = 18  
greet("David", 22)    # overrides default
```

Types of Arguments

- Variable Length Arguments
 - `*args` → for multiple positional arguments
 - Collects extra positional arguments into a tuple

```
def add_numbers(*args):  
    return sum(args)
```

```
print(add_numbers(1, 2, 3, 4)) # 10
```

Types of Arguments

- Variable Length Arguments
 - `**kwargs` → for multiple keyword arguments
 - Collects extra keyword arguments into a dictionary

```
def print_info(**kwargs):  
    for key, value in kwargs.items():  
        print(f"{key}: {value}")  
  
print_info(name="Emma", age=21, country="Germany")
```

Calling/Invoking a Function

- To use a function that has been defined before, a *function call* statement needs to be written.
- It usually follows the following format:
 - <Function name>(<value to be passed to the argument>)

```
def cube(number):  
    result = number ** 3  
    return result  
  
print(cube(4)) #Passing a literal as an argument  
  
num = 5  
print(cube(num)) #Passing a variable as an argument  
  
doubleOfCube = 2 * cube(6) # Using it in an expression  
print(doubleOfCube)
```

Scope of Variables

- Scope refers to part(s) of a program within which a name is legal and accessible.
- Analogy : The Deutschland Ticket works for anyone who is traveling in Germany. If the person tried to use it in Italy it wouldn't work as it is not valid. The scope of the ticket is Germany.

Global Scope

- A name/ variable having a global scope in Python is *usable inside the whole program and all blocks(functions, other blocks) contained within the program.*
- Analogy : The Interrail Pass provides access to trains and ferries in 33 European countries. This scope would be a global scope when compared to the Deutschland ticket which can only be used in Germany

Local Scope

- A name/ variable declared in a function body is said to have local scope. It can be used *only within this function and the other blocks contained under it*
- Analogy : Deutschland ticket working only in Germany and not outside.

Keyword global

- The global keyword in Python is a declaration that tells the interpreter that a variable inside a function refers to a global variable
- Without the keyword “global” any assignments inside the functions creates local variables by default

Why do we need global?

- Python follows the LEGB rule for variable resolution:
 - Local – inside the current function
 - Enclosing – in any outer function
 - Global – at module level
 - Built in – built in names

```

message = "Hello World"

def change_message():
    message = "Hello Python" # Creates a new local variable
    print(f"Inside function: {message}")

change_message() # Output: Inside function: Hello Python
print(f"Outside function: {message}") # Output: Outside function: Hello World

message = "Hello World"

def change_message():
    global message
    message = "Hello Python" # Modifies the global variable
    print(f"Inside function: {message}")

change_message() # Output: Inside function: Hello Python
print(f"Outside function: {message}") # Output: Outside function: Hello Python

```

Practice Programs

- Write a function to print all the even numbers from 1,20
- Write a function to calculate the area of a square asking the user for an input value
- Write a function to check if a number is a prime number

Solutions

```
def even_numbers():
    for i in range(2,21,2):
        print(i, end = " ")

even_numbers() #Output : 2 4 6 8 10 12 14 16 18 20

def is_prime(number):
    for i in range(2,number):
        if number%i == 0:
            return False
        else:
            return True

number = int(input("Enter a number: "))
print(is_prime(number))

def area_square(side):
    return side*side

side = int(input("Enter the side length of a square : "))
print("The area of the square is : ", area_square(side))
```

I/O Functions

```
# --- File I/O for mission logs ---

# Writing mission logs
with open("mission_log.txt", "a") as log:
    log.write("Event: " + event + "\n")

# Reading updates from Earth
with open("mission_updates.txt", "r") as log:
    print("=== Mission updates ===")
    print(log.read())
```

- To implement these we're going to learn Functions and File I/O in this lecture

File I/O

- Why do we need to use files?
 - Unlike variables in memory, files Store data permanently
 - Users can read existing data from files
 - Examples : saving logs, settings, student records

Opening a File

- Use `open(filename, modes)`
- Common modes:
 - “r” → read (default)
 - “w” → write (overwrites a file , creates a file if it doesn't exist)
 - “a” → append (adds to a file)
 - “b” → binary mode
- If *with open(filename, mode) as f* is used then a user doesn't need to explicitly close a file

Writing Data

```
f = open("notes.txt", "w")
f.write("Line 1\n")
f.write("Line 2\n")
f.close()
```

Reading Data

- There are multiple functions that allow a user to read the data from a file
 - `read()` → reads the entire file as a single string
 - `readlines()` → reads all the lines into a list of strings
 - `readline()` → reads one line at a time

Reading Data

```
f = open("notes.txt", "r")
print(f.read())      # whole file
f.close()
```

```
f = open("notes.txt", "r")
line1 = f.readline()
line2 = f.readline()
print(line1, line2)
f.close()
```

```
f = open("notes.txt", "r")
lines = f.readlines()
print(lines)      # ['line1\n', 'line2\n']
f.close()
```

Practice

- Write into a text file a small text. From the same file read the text written and remove all the numbers from it
- Research the working of overwriting and appending when a text file is opened in “w” mode and “a” mode

Modules

- A module in Python is a file containing Python code – definitions of variables, functions and classes – that serves as a self – contained unit importable into other Python scripts for reuse
- Modules as the name suggests promote modularity, help break down complex projects into smaller manageable chunks for easier maintenance and improved structure

Benefits of using Modules

- Code Reusability : Centralizes code for reuse across different projects or script sections
- Organization : Keeps the code organized and structured.
- Reduced Duplication : Changes made in modules are seen across the project, minimizing repetitive code

Types of Modules

- Standard Library Modules : Built in modules like math, datetime, os, random, json. These solve common programming challenges and come with Python by default
- Third-Party Modules : Community created modules such as requests(for HTTP), numpy(for scientific computing). Installable via pip from PyPI
- Custom/User-Defined : Python files created by developers to encapsulate project-specific functionalities.

Importing Modules

- The user can choose to import either the entire module or certain functions from the module
- To import the module : `import <Module Name>`
- To import functions : `from <Module Name> import <Function name>`

Importing Modules

```
import math
print(math.sqrt(49))
7.0
```

```
from math import sqrt
print(sqrt(25))
5.0
```

```
import random
num = random.randint(1,10)
print(num)
4
```

```
from random import choice
colors = ['red', 'green', 'blue']
print(choice(colors))
green
```

Summary

- Functions
- Modules
- Any Questions?